Comparative study of symbolic execution tools applied to vulnerability detection

André Cardoso
School of Technology
Polytechnic Institute of Cávado and Ave
Barcelos, Portugal
a18848@alunos.ipca.pt

Abstract

This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms

component, formatting, style, styling, insert

I. INTRODUCTION

In the digital era, it is evident that most individuals interact with devices running software that may be susceptible to exploitation. A fundamental principle in cybersecurity is that all software contains vulnerabilities; therefore, its security posture depends primarily on the difficulty of detection. The risk of attack correlates with the diligence of security teams in remediation efforts and the potential incentives available to attackers.

As such, many tools to find vulnerabilities in an automated fashion have been developed and distributed, each with their own set of pros and cons. The tools in question can be categorized as static or dynamic, and each on their own use different technics to accomplish different results. SAST (Static Application Security Testing) ultimately relies on some variety of pattern matching, data-flow analysis, and symbolic execution [1], while DAST (Dynamic Application Security Testing) excels at mimicking realistic attacks and threats [2].

The objective is a comparison of two symbolic execution tools and see how they compare in terms of performance and their ability to detect vulnerabilities. To achieve this comparison, the tools must run on projects that are preferably coded in the same language, but if these tools do not support the same set of languages, the projects should have similar architecture or business logic/intent (e.g.: online stores, accounting services, ...). These tools will also be compared on how they can be tuned in order to achieve better results.

For a better overview of how the comparison will be performed, here follows a set of questions and metrics used to measure the tools:

- Time to test a project
- CPU usage while testing a project
- Memory usage while testing a project
- Analysis of vulnerabilities detection (Youden's Index)
- Complexity of the vulnerabilities detected
- Ability to find vulnerabilities out-of-the-box
- Ability to find vulnerabilities with proper manipulation
- Effort of tuning the tool

In sum, the expected output is a clear comparison of tools that can assist developers and software companies in keeping code secure proactively and explain what differs between them and why one should pick one over the other(s). These is how we intend to answer the following research questions:

- **Which tool is more effective at finding vulnerabilities?**
- **Which tool is more efficient in terms of performance?**

II. LITERATURE REVIEW

The security of an application showed its first impacts around the beginning of the century with the increasing popularity of the WWW (World Wide Web) and the enablement of user-fuelled websites (like social media). This rapid evolution paired with the common lack of knowledge of the users but also of the developers caused the birth of Denial of Service and Cross-Site Scripting attacks among others [3].

A. Application Security Testing

AST (Application Security Testing) includes every procedure that evaluates software security, these methods may include manual or automatic testing; manual testing may perform in the form of penetration tests, but automatic testing has 2 common variants, static (SAST) and dynamic (DAST).

Static Application Security Testing is a white-box approach to software security where we have access to source code and through analysis a developer can attempt to find vulnerabilities in it; its practice is also commonly described as finding code errors.

B. SAST - Static Application Security Testing

Expanding on the white-box technique, most vulnerabilities are due to developer's lack of awareness and/or bad programming practices [4] in such a way that OWASP Foundation [5] defines it "as part of a Code Review", a "method of computer program debugging that is done by examining the code without executing the program", and also a good technic to help find:

- "Syntax violations"
- "Security vulnerabilities"
- "Programming errors"
- "Coding standard violations"
- "Undefined values"

1) Symbolic Execution: Inside SAST's set of technics, introduced by [6], we find symbolic execution engines which can help protect systems that range from modern SoC (System-on-Chip) designs and hardware Trojans [7], to binary-level security analysis such as malware comprehension [8] and even a symbiosis between symbolic and concrete execution [9]. All these tools share a common requirement: any instruction set ranging between machine code to source code. These tools excel by mimicking the execution of the analysed software, controlling each value the input influences in the form of symbolic values or variables.

However, a more in-depth look at symbolic execution unveils some obstacles, such as path explosion, constraint solving, environment modelling and others. These symbolic values are allowed to be anything at a first stage, and its value is deciphered further down in the execution path, reducing the possibilities at every branch the execution encounters. The branching happens after performing constraint solving, a very time expensive operation especially when the tested source is of high complexity [10].

Constraint solving occurs at every moment where a variable value impacts the flow of execution or even once a symbolic variable takes part in any arithmetic computation. This escalates once these computations become of a non-linear nature, such as non-linear expressions and non-linear floating-point computations [11].

Path explosion is one concern found in many high performance problems: the input is sufficiently complex for any sort of resource - be it time, memory, or even processing capacity - to escalate tremendously and becoming unreasonable in a standard environment. When it comes to symbolic execution, a new path is encountered every time a constraint is solved; from that point on, that constraint is a path condition. To better understand this concept, here's an example using the tool *KLEE* according to [12, p. 3]:

> "When KLEE detects an error or when a path reaches an exit call, KLEE solves the current path's constraints (called its path condition) to produce a test case that will follow the same path when rerun on an unmodified version of the checked program (e.g., compiled with gcc)."

Environment modelling is another very common problem in software analysis techniques given how much software development relies on third-party libraries today. This problem escalates whenever these dependencies are packaged as binaries, or the sources are 'super-optimized', the paths exploration and consequently the constraint solving may fail which may result in false positives or false negatives [12].

III. SYMBOLIC EXECUTION TOOLS

There are a couple of tools available that perform symbolic execution in numerous ways with different focus and strengths. A good agglomerate of these tools is maintained in a GitHub repository owned by Luckow [13]. There we find symbolic execution tools focused on binaries and various programming languages like Java, JavaScript, or even Rust, but also portable platforms like WASM and LLVM.

This chapter focuses on the tools that were considered for this work, and the reasons behind the final choice (section III-A). Then, the chosen tools, KLEE and Owi, are described in more detail. For both tools, installation instructions, features, and usage examples are provided (sections III-B and III-C, respectively). Finally, a comparison between both tools is presented in section III-D.

A. Engines Availability

In the endeavour of searching for tools capable of symbolic execution many attempts to have a functional installation of the available alternatives were done, but not all were successfull.

KLEE and Owi are both far better when it comes to the available documentation, and initial learning curve; the users are not bombarded with a codebase and a single 'README' file, instead they have either multiple README's for each subcommand [14] or a large number of papers and websites with tutorials, examples and information on the tool features [12, 15–18].

Given the current software paradigm, where performance is important not only in the matter of software runtime but also when it comes to software development, maintenance, and portability, tools that apply to technologies such as LLVM and WASM are interesting research targets. Therefore, we are looking at KLEE and Owi, which propose solutions for each tech stack, respectively.

B. KLEE

KLEE explores the LLVM infrastructure rather successfully and with speed; however, this engine bottlenecks quite considerably when it is applied on larger scale projects. This is due to its space management, given how it tries to keep all relevant information about the current states. It is also unable to handle dynamically generated code, concurrent execution and struggles with modelling network and peripherals, although most peers share these weaknesses [19]. The LLVM project supports many different languages like Haskell, Rust and Python, but our test cases will focus on C/C++ programs [20].

LLVM has been referenced a few times in this document, but a brief explanation is in order. LLVM is not an acronym, but the full name of the project. Despite the name, it has nothing to do with virtual machines. LLVM is "a collection of modular and reusable compiler and toolchain technologies". It is also praised for its versatility, flexibility, and reusability. For more information about LLVM, please refer to their website [20].

Reference [12] describes KLEE as being in both static and dynamic techniques of testing: every symbolic execution engine is static by definition, but KLEE becomes dynamic through its outside environment interaction features.

1) Features: KLEE is packaged with multiple CLIs (see **??**) that contribute to the user experience, but the main command that runs the symbolic execution engine is `klee`. Another relevant command is `ktest-tool`, which describes the generated test cases (`.ktest` files) in a human-readable format.

2) Instrumentation: The KLEE web page contains a tutorials section to help get familiarized with the tool. These tutorials contain samples and instructions on how to properly set up a C/C++ source file, let's call it code instrumentation, and how to run and analyse the results [16, 17].

The first, and most basic example, shows how one can mark a variable as symbolic by using the `klee_make_symbolic()` function This function takes 3 arguments; the first two will define a memory range in which the variable information is stored, the third defines a human-readable object name that identifies the symbolic variable in the generated test case files:

1) The variable memory address,
2) The variable memory size,
3) The variable name, a human-readable identifier.

Another valuable feature, is the `klee_prefer_cex()` that tells KLEE to prefer a set of values when generating test cases; this allows the user to get readable results like "aaaa" instead of hexadecimal representation of character values such as "\xff\xff\xff\xff".

3) Running KLEE: After instrumenting the source code with KLEE instructions, the next step is compiling it to LLVM. The KLEE documentation recommends the command in Listing 1 in order to have access to a more complete set of functionalities that klee provides, such as test replayability.

```
clang -emit-llvm -c -g -O0 -Xclang -disable-O0-optnone main.c
```

Listing 1: Complete clang command

The compilation command should return an object with extension '*.bc*' containing the LLVM representation of the program. This file is in the correct format for KLEE, and therefore we can now test it by running the command on Listing 2.

```
klee main.bc
```

Listing 2: Running klee on a program

Running KLEE on a program results in file outputs, the `klee-out-n` and `klee-last` folders are created. For `klee-out-n`, `n` is the zero-indexed counter of the run, and `klee-last` is a symbolic link to the most recent run folder; by the second run of KLEE, `klee-last` points to `klee-out-1`. The folders created by KLEE contain all the generated test cases, error reports, and other useful information about the execution. Each generated test case is stored in a file with extension '*.ktest*' and can be analysed using the `ktest-tool` command (see Listing 3).

```
$ ktest-tool klee-last/test000001.ktest
ktest file : 'klee/klee-last/test000001.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'my signed integer'
object 0: size: 4
object 0: data: b'\x00\x00\x00\x00'
object 0: hex : 0x00000000
object 0: int : 0
object 0: uint: 0
object 0: text: ....
```

Listing 3: Analyzing a ktest file

C. Owi

Reference [21] describes Owi as a toolkit that aggregates functionality to assist WebAssembly developers, one of which enables the compilation of C code to WASM and running their symbolic interpreter on top of it.

WebAssembly is a portable binary instruction format for executable programs, designed to be a compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications. WebAssembly is designed to be fast, efficient, and secure, making it an ideal choice for running complex applications in web browsers and other environments [22, 23].

1) Features: Owi being a toolchain designed for the development of WASM, it provides a set of subcommands but this study will focus on the ones related to symbolic execution, which is the '*sym*' subcommand. To be able to use the symbolic interpreter (`owi sym`), we need to also install a restraint solver. Owi supports any Smt.ml supported solver, but the default solver is Z3.

2) Instrumentation: Unlike KLEE, that uses the variable address only and ignores the type, Owi needs to be told the symbolic variable type, through the available functions:

- `owi_i32()` - to define 32-bit integer values, with 8-bit and 64-bit variants;
- `owi_f32()` - to define 32-bit floating-point values, with a 64-bit variant;
- `owi_char()` - to define character values;
- `owi_bool()` - to define boolean values.

The symbols are identified by order of creation - if we define all positions of an array as symbolic, each position symbolic identifier would correspond to their array position, but if any other symbol is defined before the identifiers would shift by the number of predefined variables.

To run Owi on a simple sample we must include the Owi library in the source code so it is possible to find Owi function implementations and generate a valid WebAssembly module.

Owi also produces a `owi-out/test-suite` folder and inside it a `testcase-N.xml`, where `N` is an index starting on 1, that describes the input value for a symbolic variable for that test case, and a `metadata.xml` that details the arguments of the owi executation such as the source code language, which Owi subcommand produced the file, the input program file ('*poly.c*') and other details on the execution. The generated test cases are relevant to found issues, Owi does not generate test cases on each traced path like KLEE does.

3) Running Owi: Once the sources are ready, we need to compile the source code so we can run `owi sym`; the default compilation command that Owi uses when running `owi c` on a C source file, can be found by running `owi c --debug` on invalid C files and the tool outputs the command in Listing 4.

```
clang -O3 --target=wasm32 -m32 -ffreestanding \
    ↪ --no-standard-libraries -Wno-everything -flto=thin \
    ↪ -Wl,--entry=main -Wl,--export=main -Wl,--lto-O0 \
    ↪ -Wl,-z,stack-size=8388608 \
    ↪ -I $HOME/.opam/default/share/owi/libc -o main.wasm \
    ↪ $HOME/.opam/default/share/owi/binc/libc.wasm \
    ↪ main.c
```

Listing 4: Owi abstracted compilation command (clang).

After compiling `main.c`, the resulting `.wasm` object can be tested with Owi using the command on Listing 5.

```
owi sym main.wasm
```

Listing 5: Running owi on a compiled program.

Executing the `sym` subcommand generates some files inside the folder 'owi-out' created on the current working directory: `owi sym` only generates test case files named 'testcase-n.xml' being n the number of results detected and starting from 1.

D. Feature Comparison

Not only it is important to understand how well a tool performs, but also how flexible it can be in the different scenarios it may face.

The first section of Table I describes the installation process and ease-of-use of each tool. Installation is scored in two dimensions: speed and complexity. KLEE supports a quick start using Docker, which is very fast to set up but may not be the best option for everyone. The recommended installation process is more complex and time-consuming, which is reflected in the scores. Owi doesn't offer a docker image.

The second section of Table I describes features and heuristics supported by each tool; heuristics refers to software vulnerabilities/errors, if they are supported by the tools and how these heuristics are defined. This is analysed in higher detail in section IV.

The last section of Table I aggregates information from KLEE and Owi documentation on SAT and SMT solvers. While SAT solvers handle *boolean satisfiability*, SMT expand on this concept into general formulas[24]. Examples of SMT solvers are STP and Z3 but there are also wrappers like MetaSMT and frontends as Smt.ml [25–28]. MiniSat is a SAT solver used by STP as the underlying SAT solver and Smt.ml also has MiniSat in its support roadmap [28, 29].

*F*eature	KLEE	Owi
Installation speed	5 (2)	2
Installation complexity	1 (4)	3
Ease-of-use (begginer)	4	2
Ease-of-use (experienced)	4	4
Heuristics support	Larger	Reduced
Heuristics definition	Hardcoded	Hardcoded
Supports E-ACSL	NO	YES
Supports MetaSMT	YES	NO
Supports STP	YES	NO
Supports Z3	YES	YES
Supports metaSMT solvers	YES	NO
Supports Smt.ml solvers	NO	YES
Supports MiniSat	YES	NO

TABLE I: Feature comparison

IV. EXPERIENCES AND RESULTS

In this chapter, multiple experiences using both KLEE and Owi tools will be described and analysed. We will start by describing the test environment in section IV-A, and then proceed to describe each experience in its own section. Finally, a summary of all results and an overview of the experiences will be presented in section IV-F. Each experience will showcase their capabilities and limitations.

ADALogics [30] describe themselves as "a world-class player in software security" and as such they have many interactions in software security applications, such as symbolic execution. One of these interactions is a blog on KLEE where they go through the installation and usage of the tool to find bugs. On this blog, ADALogics [15] presents 4 videos to demonstrate KLEE set of features and in video 3 they discuss vulnerabilities of the types:

- Global Buffer Overflow (Sample 6 - Global Buffer Overflow),
- Stack-based Buffer Overflow (Sample 7 - Stack-based Buffer Overflow),
- NULL pointer dereference (Sample 8 - NULL pointer dereference),
- Use after `free` (Sample 9 - Use after free).

Sections IV-B to IV-E will cover these vulnerabilities using both KLEE and Owi by instrumenting each source in the intended way.

A. Test Environment

To increase the results' credibility, all tests were run on the same machine, therefore sharing the same resources. The specifications of such machine consisted in a 12th Gen Intel Core processor (i7-1255U) of 10 cores (and a total of 12 threads) with a max clock speed of 4.70 GHz, and 20 GB (4 + 16) of DDR4 3200 MHz RAM.

This is the parent system setup with a Windows 11 installation; however, the test environment was created in a WSL environment within this machine which shares 4 cores and 8 GB of RAM available.

For simplicity of testing, all samples were organized by folders and subfolders: the parent folder aggregates by sample and child folders aggregates files by tool, KLEE or Owi.

B. Sample 6 - Global Buffer Overflow

On this section, we will be covering an example of a Global Buffer Overflow vulnerability and how these problems can be detected using KLEE and Owi.

On **??**, we can see how the `test1` input variable `arr` is safely marked as symbolic with **????**, using the KLEE instrumentation methods. **??** makes the array safe for string operations like '`strcmp`' on **??**.

The second instrumentalization is done on Listing 6. Because the symbolic input is an array, each position must be initialized (see sections IV-B to IV-B) and because this is an array of character values (a string) we're telling Owi the string value is suffixed by '\0' using `owi_assume` function and passing the condition `arr[9] == '\0'` as its argument (section IV-B). The string is 'closed' so that it is safe for string operations, such as '`strcmp`' on section IV-B, and it isn't reported as a problem.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int global_arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int test1(char *buf)
{
  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    i += 20;
  }
  return global_arr[i];
}

int main(int argc, char const *argv[])
{
  char arr [10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test1(arr);

  return 0;
}
```

Listing 6: Sample 6 instrumented for Owi

Starting with klee execution, the results are found very quickly and only 6 paths are traced, one of which is partial and an error detection. Listing 7 also shows on section IV-B the type of error ("`memory error: out of bound pointer`") and on which file and line the error occurred ("`main.c:14`"). All unique errors generate a test case of its own and the one detected for this sample is expressed in detail on Listing 8.

On the other hand, owi displays "`All OK`" after scanning the sources stating it did not find the vulnerability.

```
KLEE: ERROR: main.c:14: memory error: out of bound pointer
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 12493
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 7: Sample 6 - KLEE output

```
ktest file : './sample6/klee/klee-last/test000003.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 8: Sample 6 - KLEE test case for the detected error

C. Sample 7 - Stack-based Buffer Overflow

Differently from Sample 6 - Global Buffer Overflow, Stack-based Buffer Overflow vulnerabilities look at scoped and dynamically allocated variables like the ones on line 8 in both KLEE and Owi samples (**??** and listing 11); the samples' instrumentation is identical to what is seen on Sample 6 - Global Buffer Overflow.

Running KLEE results in a very similar result to that seen on section IV-B, once again 6 generated tests, one of which triggered an execution error shown in detail on Listing 10. This time, the error occurs on **??** of file main (**??**). Owi fails to find the vulnerability, again.

```
KLEE: NOTE: KLEE: ERROR: main.c:26: memory error: out of bound pointer
now ignoring this error at this location

KLEE: done: total instructions = 12566
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 9: Sample 7 - KLEE output

```
ktest file : './sample7/klee/klee-last/test000003.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 10: Sample 7 - KLEE test case for the detected error

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int test2(char *buf)
{
  int *dyn_mem_arr = malloc(sizeof(int) * 10);
  dyn_mem_arr[0] = 0;
  dyn_mem_arr[1] = 1;
  dyn_mem_arr[2] = 2;
  dyn_mem_arr[3] = 3;
  dyn_mem_arr[4] = 4;
  dyn_mem_arr[5] = 5;
  dyn_mem_arr[6] = 6;
  dyn_mem_arr[7] = 7;
  dyn_mem_arr[8] = 8;
  dyn_mem_arr[9] = 9;

  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    i += 20;
  }

  int return_value = dyn_mem_arr[i];
  free(dyn_mem_arr);
  return return_value;
}

int main(int argc, char const *argv[])
{
  char arr[10];
  for (int i = 0; i < 10; i++)
  {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test2(arr);
  return 0;
```

```
42 }
```

Listing 11: Sample 7 - Owi instrumentation

D. Sample 8 - NULL pointer dereference

To test NULL pointer dereference bugs, a small string array (‘`char *global_string[]`’ on section IV-D) is initialized and used in function ‘`test3`’ where on section IV-D will trigger an error when the input is favorable.

After the samples are instrumented, the tools are run and a pattern begins to show where KLEE detects the vulnerability but Owi does not. See **??** and listing 12 for KLEE and Owi instrumented samples, respectively, Listing 13 for KLEE terminal output and Listing 14 for the generated test case.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

char *global_string[] = {
  "string1",
  "string2",
  "string3",
  NULL,
};

int test3(char *buf)
{
  int i = 1;
  if (strcmp(buf, "BUG!") == 0)
  {
    i = 3;
  }

  char c = *(global_string[i]);
  if (c == 's')
    return 0;
  return 1;
}

int main(int argc, char const *argv[])
{
  char arr[10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
  owi_assume(arr[9] == '\0');

  test3(arr);
  return 0;
}
```

Listing 12: Sample 8 - Owi instrumentation

```
KLEE: ERROR: main.c:21: memory error: out of bound pointer
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 12549
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 13: Sample 8 - KLEE output

```
ktest file : './sample8/klee/klee-last/test000003.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 14: Sample 8 - KLEE test case for the detected error

E. Sample 9 - Use after free

Finally, 'Use after free' runtime errors can also become very hard to detect by the human eye when code complexity increases; '`test4`' function presents a simple case of this vulnerability to showcase KLEE and Owi capabilities. **??** and listing 18 illustrate instrumented sources for each symbolic execution engine.

KLEE detected the bug on **??**, generated 6 tests, and distinguished the error from the ones on sections IV-B to IV-D. KLEE's output (Listing 15) shows that the memory error is of type '`double free`' on section IV-E, and the generated test case (Listing 16) shows the expected input value.

```
KLEE: ERROR: main.c:14: memory error: double free
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 12484
KLEE: done: completed paths = 5
KLEE: done: partially completed paths = 1
KLEE: done: generated tests = 6
```

Listing 15: Sample 9 - KLEE output

```
ktest file : './sample9/klee/klee-last/test000003.ktest'
args       : ['main.bc']
num objects: 1
object 0: name: 'arr'
object 0: size: 10
object 0: data: b'BUG!\x00\xff\xff\xff\xff\x00'
object 0: hex : 0x4255472100ffffffff00
object 0: text: BUG!......
```

Listing 16: Sample 9 - KLEE test case for the detected error

While KLEE detected the double free as any other code issue, Owi reported the problem as an internal error and its respective stack trace as displayed in Listing 17. This behaviour suggests the tool does not properly support this issue and might cause the scan to miss other issues because of this internal error.

```
owi: internal error, uncaught exception:
     Failure("Memory leak double free")
     Raised at Stdlib__Domain.join in file "domain.ml", line 296, characters 16-24
     Called from Stdlib__Array.iter in file "array.ml", line 113, characters 31-48
     Called from Owi__Wq.read_as_seq.(fun) in file "src/data_structures/wq.ml", line 17, characters 4-16
     Called from Owi__Cmd_sym.cmd.print_and_count_failures in file "src/cmd/cmd_sym.ml", line 99,
    ↪ characters 10-20
     Called from Owi__Cmd_sym.cmd in file "src/cmd/cmd_sym.ml", line 130, characters 15-49
     Called from Cmdliner_term.app.(fun) in file "cmdliner_term.ml", line 24, characters 19-24
     Called from Cmdliner_eval.run_parser in file "cmdliner_eval.ml", line 35, characters 37-44
```

Listing 17: Sample 9 - Owi output

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <owi.h>

int test4(char *buf)
{
  char *var = malloc(10);
  int i = 5;
  if (strcmp(buf, "BUG!") == 0)
  {
    free(var);
  }
  free(var);

  return 1;
}

int main(int argc, char const *argv[])
{
  char arr[10];
  for (int i = 0; i < 10; i++) {
    arr[i] = owi_char();
  }
```

```
25    owi_assume(arr[9] == '\0');
26
27    test4(arr);
28    return 0;
29  }
```

Listing 18: Sample 9 - Owi instrumentation

F. Results

This section discusses **Accuracy** and **Performance** metrics such as:

Accuracy Metrics

- **TP - True Positives**
  Number of **expected** vulnerabilities **reported**.
- **TN - True Negatives**
  Number of **unexpected** vulnerabilities **not reported**.
- **FP - False Positives**
  Number of **unexpected** vulnerabilities **reported**.
- **FN - False Negatives**
  Number of **expected** vulnerabilities **not reported**.
- **Youden Index or Youden's J statistic**
  The accuracy metric used in OWASP Benchmark [31] is the following
  $J = TPR - FPR$

$$= (\frac{TP}{TP+FN})^a - (\frac{FP}{TN+FP})^b$$

$$= \frac{TP*TN - FP*FN}{(TP+FN)(TN+FP)}$$

[a] TPR - True Positive Rate
[b] FPR - False Positive Rate

Performance Metrics

- **Paths traced**
  Branches of execution created to navigate.
- **CPU (%)**
  The CPU used by the process, in percentage.
- **Memory (KB)**
  The 'maximum resident set size' (very similar to peak memory) used by the process, in kilobytes.
- **Time (s)**
  The execution time of the process, in seconds.

Accuracy metrics were extracted by manually inspecting each scan output, and performance metrics such as CPU, memory and time of execution were measured using the GNU `time` program (see Listing 19). "**Paths traced**" was also extracted manually similarly to accuracy metrics. There is also the "**Success**" metric that indicates whether the sample was successfully analysed by the test tool, without regard for results.

```
1 # to avoid confusion with bash keyword 'time'
2 # use the full path given to you by `which time`
3
4 time -v klee main.bc
5
6 time -v owi sym main.wasm
```

Listing 19: Extraction of performance metrics

After executing both engines on all test samples and extracting the results from each of them, Tables II and III were created. With these metrics properly formatted, calculating the "**Youden Index**" for both tools is easier, higher is better.

It is apparent how KLEE detected much more vulnerabilities than Owi, which plays in its favour, and the calculations on equations (1) and (2) reflect this statement, KLEE scores approximately 54% while Owi stands on 29% of bug detection accuracy.

If we look at the Youden Index interpretation guide from OWASP Benchmark [31] shown in Figure 1, we can see that KLEE scores highly (87.5%) in the *y* axis (TPR), while Owi scores poorly (28.5%). Both tools score low on the *x* axis (FPR), with KLEE at 33.3% and Owi at 0%. This means that KLEE is very good at detecting vulnerabilities, but also has a higher rate of false positives, while Owi is not very good at detecting vulnerabilities, but has a low rate of false positives. A larger set of samples would be needed to draw more concrete conclusions.

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	2	0	0	0
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES[a]	0	0	1	1
6	YES	1	0	0	0
7	YES	1	0	0	0
8	YES	1	0	0	0
9	YES	1	0	0	0

TABLE II: KLEE accuracy metrics

[a]KLEE was able to run, but it wasn't able to make use of Frama-C's E-ACSL capabilities

Sample	Success	TP	TN	FP	FN
1	YES	0	1	0	0
2	YES	0	0	0	1
3	YES	1	0	0	0
4	YES	0	1	0	0
5	YES	1	0	0	0
6	YES	0	0	0	1
7	YES	0	0	0	1
8	YES	0	0	0	1
9	YES	0	0	0	1

TABLE III: Owi accuracy metrics

$$J = (\frac{7}{7+1})^{*} - (\frac{1}{2+1})^{**} \approx (0.875) - (0.333) \approx 0.54 \quad (1)$$

KLEE Youden Index

$$J = (\frac{2}{2+5})^{*} - (\frac{0}{2+0})^{**} \approx (0.285) - (0) \approx 0.29 \quad (2)$$

Owi Youden Index

* TPR - True Positive Rate

** FPR - False Positive Rate

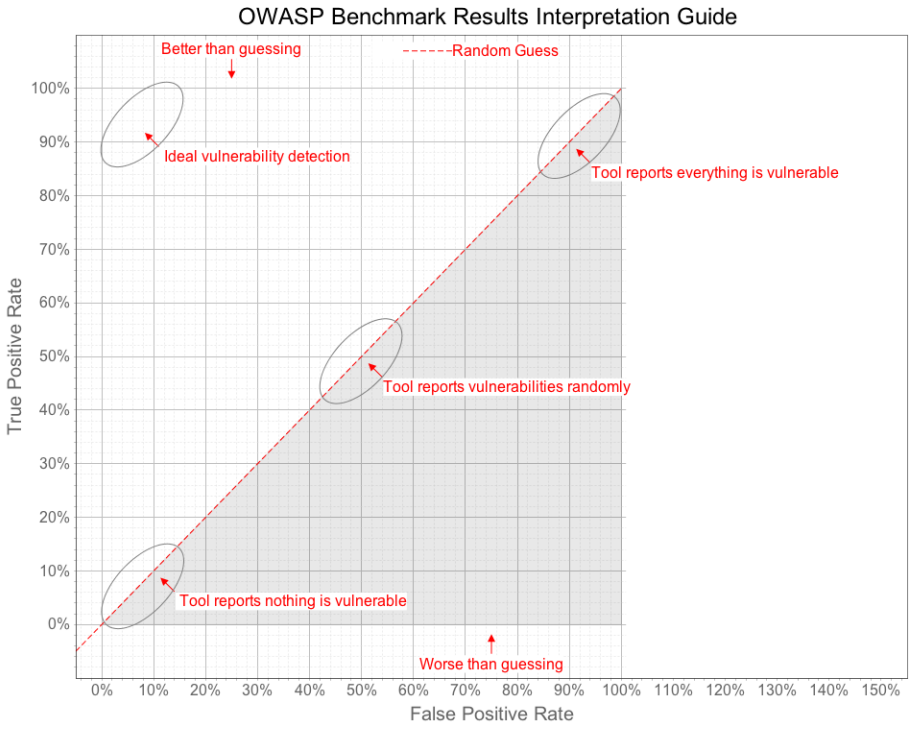

Fig. 1: OWASP Benchmark Scoring Guide - Youden Index interpretation [31]

By running KLEE and Owi on the same set of samples, some differences are visible in their outputs and in their performances (see Section IV-F). KLEE was overall slower, with the only exception being the primes example, but was also the example that consumed less CPU but also Memory on all samples.

Unfortunately, Owi does not seem to output information regarding the travelled states, even after looking at debug logs (which depict execution instructions, but nothing was found to support a number of traversed paths).

An example of how the performance between these tools differ is with sample 3, a test where KLEE traces 306 paths. KLEE reported results consuming nearly half the CPU resources and approximately 73% of the memory usage and 35% of the time spent, that Owi required to run and return no results on the sample.

Sample 2 shows an even greater difference; KLEE spends nearly half the CPU and 19% of the memory usage when compared to Owi, and does that in 1% of the time it took Owi to analyse the same source. This scenario might be a clear indicator of Owi limitations.

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	3	99	98'156	0.29
2	YES	6'675	101	139'680	12.93
3	YES	305	93	99'152	0.72
4	YES	99	98	99'504	2.93
5	YES	1	81	98'036	0.39
6	YES	5	102	98'920	0.30
7	YES	5	87	97'536	0.37
8	YES	5	96	98'248	0.33
9	YES	5	96	99'236	0.32

TABLE IV: KLEE performance metrics

[a]Sample number
[b]The best out of five runs

#[a]	Success	Paths traced	CPU (%)	Memory (KB)	Time (s)[b]
1	YES	N/A	108	110'000	0.05
2	YES	N/A	200	725'892	901.51
3	YES	N/A	199	134'928	2.10
4	YES	N/A	204	145'676	9.90
5	YES	N/A	205	151'800	11.13
6	YES	N/A	116	120'172	0.06
7	YES	N/A	115	119'172	0.06
8	YES	N/A	118	118'220	0.05
9	YES	N/A	123	121'040	0.05

TABLE V: Owi performance metrics

[a]Sample number
[b]The best out of five runs

REFERENCES

[1] R. Croft, D. Newlands, Z. Chen, and A. M. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," *International Symposium on Empirical Software Engineering and Measurement*, 10 2021. [Online]. Available: http://eprints.whiterose.ac.uk/95628/

[2] M. Sharma, "Review of the benefits of dast (dynamic application security testing) versus sast," *International Journal of Management and Engineering Research*, vol. 1, pp. 1–4, 6 2021. [Online]. Available: http://www.ijmer.org/index.php/journal/article/view/2

[3] A. Hoffman, *Web Application Security*. O'Reilly Media, Inc., 2024.

[4] M. Alenezi and Y. Javed, "Open source web application security: A static analysis approach," *IEEE*, pp. 1–5, 2016.

[5] OWASP Foundation. (2023) OWASP DevSecOps Guideline - v-0.2 | 2-a - Static Application Security Testing. [Online]. Available: https://owasp.org/www-project-devsecops-guideline/latest/02a-Static-Application-Security-Testing

[6] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, pp. 385–394, 7 1976.

[7] A. Vafaei, N. Hooten, M. Tehranipoor, and F. Farahmandi, "Symba: Symbolic execution at c-level for hardware trojan activation," *Proceedings - International Test Conference*, pp. 223–232, 2021.

[8] R. David, S. Bardin, T. D. Ta, J. Feist, L. Mounier, M. L. Potet, and J. Y. Marion, "Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis," *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, SANER 2016*, vol. 1, pp. 653–656, 5 2016.

[9] F. Gritti, L. Fontana, E. Gustafson, F. Pagani, A. Continella, C. Kruegel, and G. Vigna, "Symbion: Interleaving symbolic with concrete execution," *2020 IEEE Conference on Communications and Network Security, CNS 2020*, 6 2020.

[10] G. Zhang, Z. Chen, Z. Shuai, Y. Zhang, and J. Wang, "Synergizing symbolic execution and fuzzing by function-level selective symbolization," *Proceedings - Asia-Pacific Software Engineering Conference, APSEC*, vol. 2022-December, pp. 328–337, 12 2022.
[11] D. Kroening and O. Strichman, *Decision Procedures - An Algorithmic Point of View*, 1st ed. Springer Berlin, Heidelberg, 4 2008.
[12] C. Cadar, D. Dunbar, and D. Engler, "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs," *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, 2008. [Online]. Available: https://purl.stanford.edu/fx501rc2898
[13] K. Luckow. (2017, 7) ksluckow/awesome-symbolic-execution. A curated list of awesome symbolic execution resources including essential research papers, lectures, videos, and tools. [Online]. Available: https://github.com/ksluckow/awesome-symbolic-execution
[14] Formal Security for Web Technologies. OCamlPro/owi: WebAssembly Swissknife & cross-language bugfinder. [Online]. Available: https://github.com/OCamlPro/owi
[15] ADALogics. Symbolic execution with KLEE: From installation and introduction to bug-finding in open source software. [Online]. Available: https://adalogics.com/blog/symbolic-execution-with-klee
[16] The KLEE Team. Tutorial One · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-function
[17] ——. Tutorial Two · KLEE. [Online]. Available: https://klee-se.org/tutorials/testing-regex
[18] ——. Building KLEE with LLVM 13. [Online]. Available: https://klee-se.org/build/build-llvm13/
[19] N. M. d. Sabino, "Automatic vulnerability detection: Using compressed execution traces to guide symbolic execution," Master's thesis, Instituto Superior Técnico, 2019.
[20] LLVM Project. The LLVM Compiler Infrastructure Project. [Online]. Available: https://llvm.org/
[21] L. Andrès, F. Marques, A. Carcano, P. Chambart, J. F. F. dos Santos, and J.-C. Filliâtre, "Owi: Performant parallel symbolic execution made easy, an application to webassembly," *The Art, Science, and Engineering of Programming*, vol. 9, 10 2024. [Online]. Available: https://hal.science/hal-04627413
[22] WebAssembly. WebAssembly. [Online]. Available: https://webassembly.org
[23] ——. Use Cases - WebAssembly. [Online]. Available: https://webassembly.org/docs/use-cases/
[24] RBC Borealis. Tutorial #9: SAT Solvers I: Introduction and applications. [Online]. Available: https://rbcborealis.com/research-blogs/tutorial-9-sat-solvers-i-introduction-and-applications/
[25] The STP Project. STP • The Simple Theorem Prover. [Online]. Available: https://stp.github.io/
[26] Microsoft Corporation. Documentation for Online Z3 Guide | Online Z3 Guide. [Online]. Available: https://microsoft.github.io/z3guide/
[27] Group of Computer Architecture (AGRA). agra-uni-bremen/metaSMT. [Online]. Available: https://github.com/agra-uni-bremen/metaSMT
[28] Formal Security for Web Technologies. formalsec/smtml: A frontend for multiple SMT solvers in OCaml. [Online]. Available: https://github.com/formalsec/smtml
[29] N. Eén and S. Niklas. MiniSat Page. [Online]. Available: http://minisat.se/
[30] ADALogics. About us | ADA Logics. [Online]. Available: https://adalogics.com/about-us
[31] OWASP Foundation. (2023) OWASP Benchmark | OWASP Foundation. [Online]. Available: https://owasp.org/www-project-benchmark/